# User Scenarios

## [User Scenario 1]

***1.1: Instructor initiates the process by setting up a section and inputs the team formation criteria.***

| **Microservice**<br>[A] - Atomic \| [C] - Composite | **Purpose**<br>Key Functionality / Operations |
|---|---|
| **Configuration of Section's Team Formation Criteria** | |
| [C] formation-config | **orchestrates** the collection and aggregation of the formation configuration for a section's team formation, publishing errors via RabbitMQ |
| [A] criteria | stores criteria weights for team formation (e.g. gpa weight, skill weight) |
| [A] project-topic | stores the project topics students can choose |
| [A] skills | stores relevant skills for the project |
| [A] error-log | stores error logs (to be renamed from "errors") |

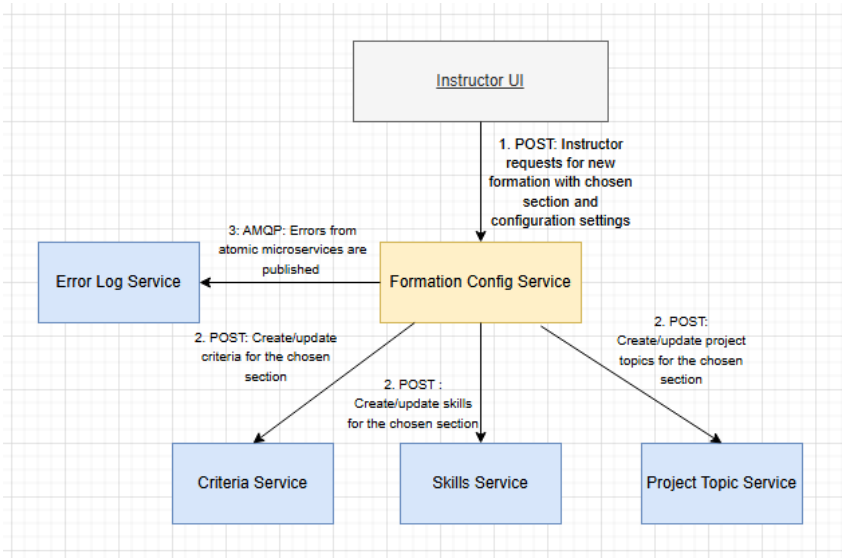


Fig 1.1 - Instructor requests for a new formation with configured settings

1. Instructor UI sends a POST request to formation-config, in a JSON with criteria weights, skills and project topics.
2. formation-config processes this JSON, then sends weights to criteria, skills to skills, project topics to project-topic.
3. formation-config publishes an error log with routing key *"formation-config-service.error"* to RabbitMQ, which will be consumed by error-log-service.

***1.2: OPTIONAL: Instructor collects student data and preferences for team formation***

| **Microservice**<br>[A] - Atomic \| [C] - Composite | **Purpose**<br>Key Functionality / Operations |
|---|---|
| **Generation of dynamic form to collect project-specific student data, notifying students via email** | |
| [C] formation-notification | orchestrates aggregation of data from enrollment, student, student-form to send emails with a form link to all students enrolled in a section by calling notification |

| | |
|---|---|
| | service, publishing errors via RabbitMQ |
| [A] enrollment | stores student enrollment details |
| [A] student-form | stores unique interfaces for each student to access their form |
| [A] student (outsystems) | retrieve student emails for notification of the form link |
| [A] error-log | stores error logs (to be renamed from “errors”) |
| [A] notification | notifies and broadcasts the form link to enrolled students |

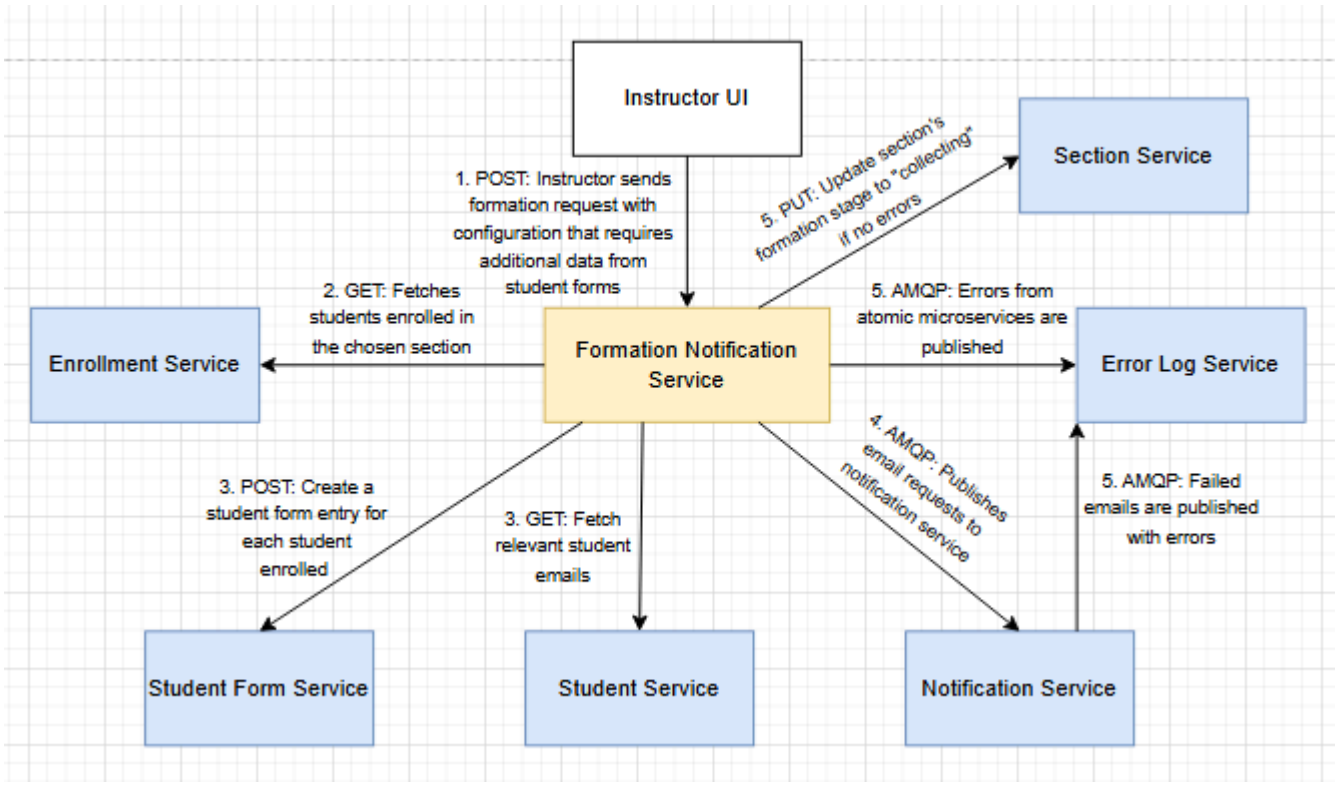


Fig 1.2 - Instructor Submits confirmed Form email and Students receives notifications via Email

1. If Instructor UI creates a team formation request that requires additional data which cannot be loaded from current student data (e.g. Buddy request for this project), sends a POST request to formation-notification for the chosen section.
2. formation-notification calls a GET request to enrollment to retrieve the enrolled students for the section.
3. formation-notification calls a POST request to student-form to create student form entries for each student enrolled and calls a GET request to collect enrolled students’ emails for notification.
4. formation-notification publishes all email requests to notification (attaches student email and link to student’s form) via routing key “notification.email”
5. Errors from formation-notification and notification are published to RabbitMQ and consumed by error-log. If formation-notification does not publish any errors, it sends a PUT request to section to update the stage to “Collecting”, which will be displayed in the instructor UI.

### *1.3: OPTIONAL: Student Submits Form*

| **Microservice**<br>[A] - Atomic \| [C] - Composite | **Purpose**<br>Key Functionality / Operations |
|---|---|

| Student UI displays dynamic student form; Form submission is stored | |
|---|---|
| [C] student-form-submission | orchestrates the collection and aggregation of data from student-form, and stores it into the various storage microservices. |
| [A] student-form-data | stores student responses from the form |
| [A] student-competence | categorises responses into their skills (to be renamed from student-competence) |
| [A] student-topic-preference | categorises student topic preference |

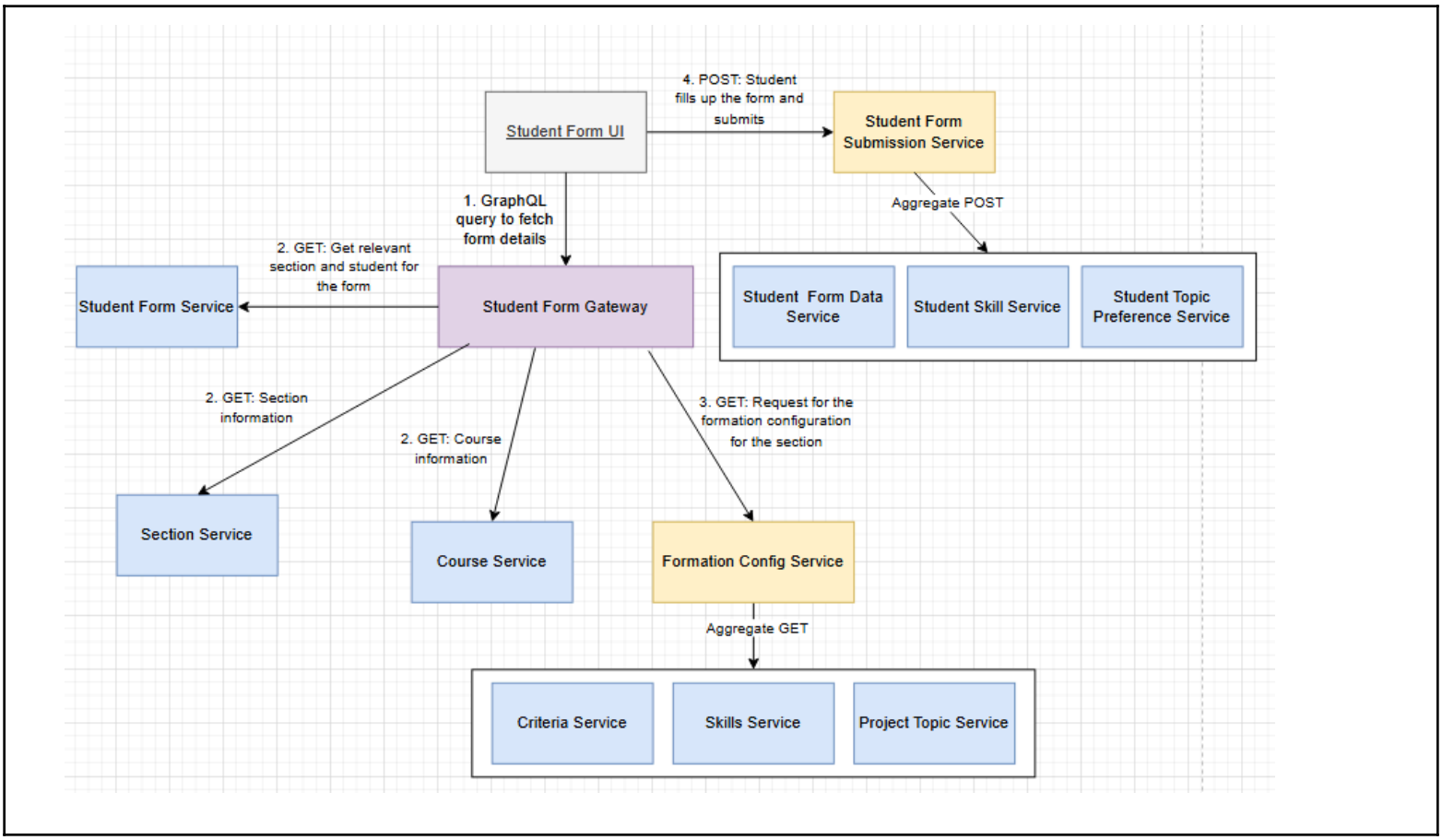


Fig 1.3 - Student responses are processed and categorised into the various atomic microservices.

1. When a student accesses student-form, the Student Form UI makes a GraphQL query to student-form-gateway to retrieve the frontend essentials to dynamically display a form.
2. student-form-gateway calls a GET request to student-form, section and course to get the main description details of the form. (e.g. section number, whether the form has been submitted, course name)
3. The data is returned to the Student Form UI, which processes the data to only display relevant input fields for the user-facing student form (E.g. If Buddy weight is set to 0, no input field for Buddy preference). After a student fills up and submits the form, the Student Form UI calls a POST request to student-form-submission as a JSON with all input details, which aggregates the POST request to atomic microservices student-form-submission processes and splits the data if required to asynchronously POST to student-form-data, student-competence and student-topic-preference.
4. student-form-submission publishes errors from its atomic services to RabbitMQ, consumed by error-log.

### *1.4: Team Generation*

| **Microservice**<br>[A] - Atomic \| [C] - Composite | **Purpose**<br>Key Functionality / Operations |
|---|---|

| Teams are generated using a solver based on student data and configuration; Teams are stored | |
|---|---|
| [C] team-formation | orchestrates the team formation process, aggregating data for team formation, running a solver to create optimised teams using Google OR-tools, then storing generated teams, publishing errors via RabbitMQ |
| [C] formation-config | aggregates and retrieves formation configuration settings |
| [C] student-profile | aggregates and retrieves student data for team formation |
| [A] student-form | stores unique interfaces for each student to access their form |
| [A] reputation | stores student's reputation score from previous projects |
| [A] team | stores generated teams of students |

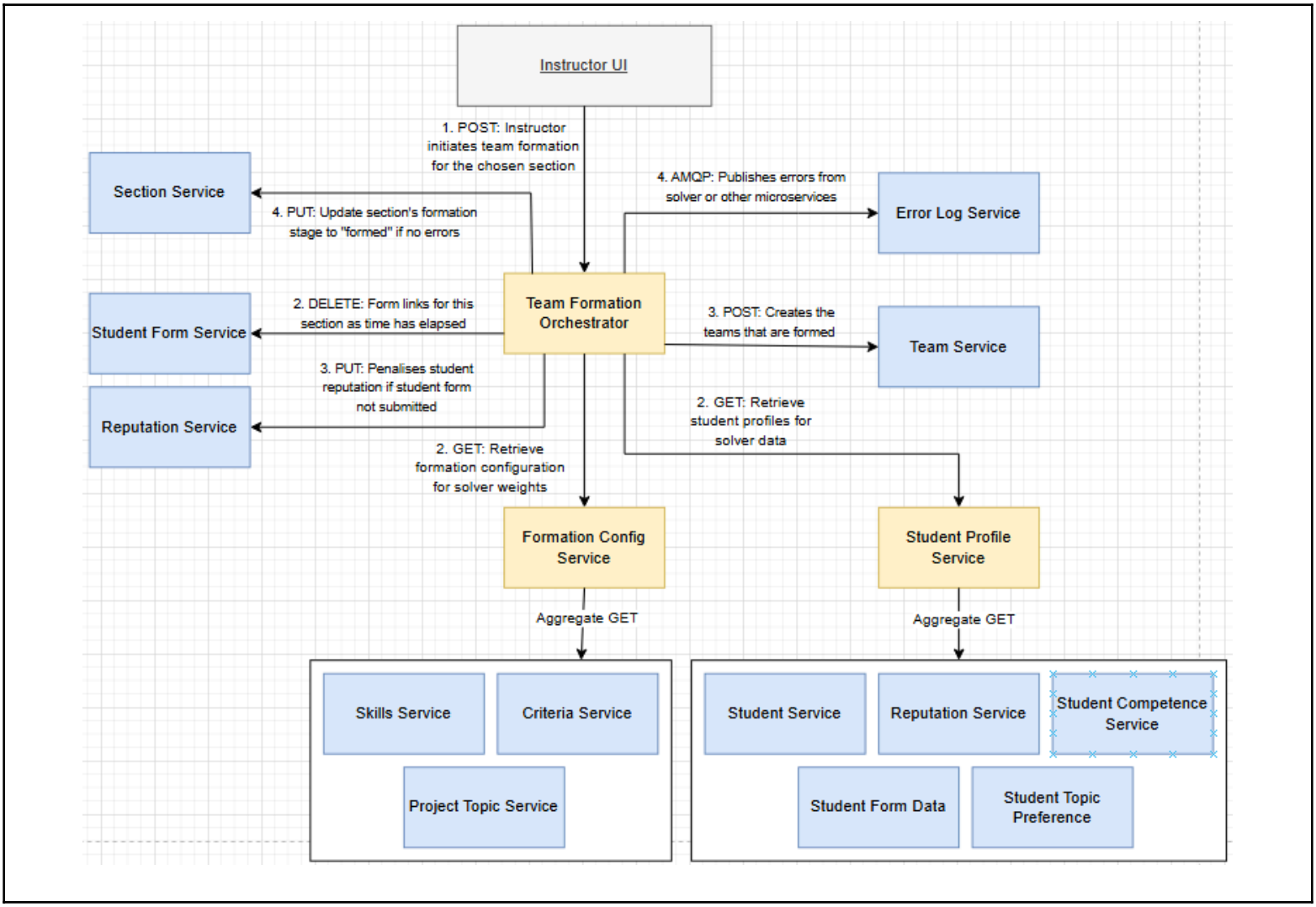


Fig 1.4 - Team Generation

1. Instructors initiate team formation after they submit formation configuration settings (if no additional data to be collected) or end student form collection (if additional data needs to be collected), sending a POST request to team-formation.
2. Deletes all student forms for the section, if any. Then, calls GET for formation-config and student-profile to aggregate and retrieve the configuration settings and student data for the solver to generate teams.
3. Penalises student's reputation score if they did not submit the forms, if any (does not affect current formation). Uses solver to generate optimised teams and POST to team to store the teams.

4. team-formation publishes any errors from the solver or other microservices called via AMQP, consumed by error-log. If team-formation does not publish any errors, it sends a PUT request to section to update the stage to "Formed", which will be displayed in the instructor UI.

## External Services

| Service Name | Description of the functionality | Link to external web pages that describe the detailed inputs/outputs |
|---|---|---|
| *Outsystems* | *Fetch/Supply student data* | *Student MS: https://personal-0wtj3pne.outsystemscloud.com/Student/rest/Student/#/* |

## [User Scenario 2]

***2: Instructor views team analytics.***

| **Microservice**<br>[A] - Atomic \| [C] - Composite | **Purpose**<br>Key Functionality / Operations |
|---|---|
| **Dashboard for team analysis and evaluation** | |
| [A] dashboard-orchestrator | orchestrates populating the dashboard through the data from relevant services |
| [A] analytics | analyses team details and outputs analytics metrics on Instructor UI page |
| [A] team | stores team groupings |
| [C] team-formation | stores the algorithm that generates the teams |
| [A] student-form | generates a unique interface/link for each student to access their form |

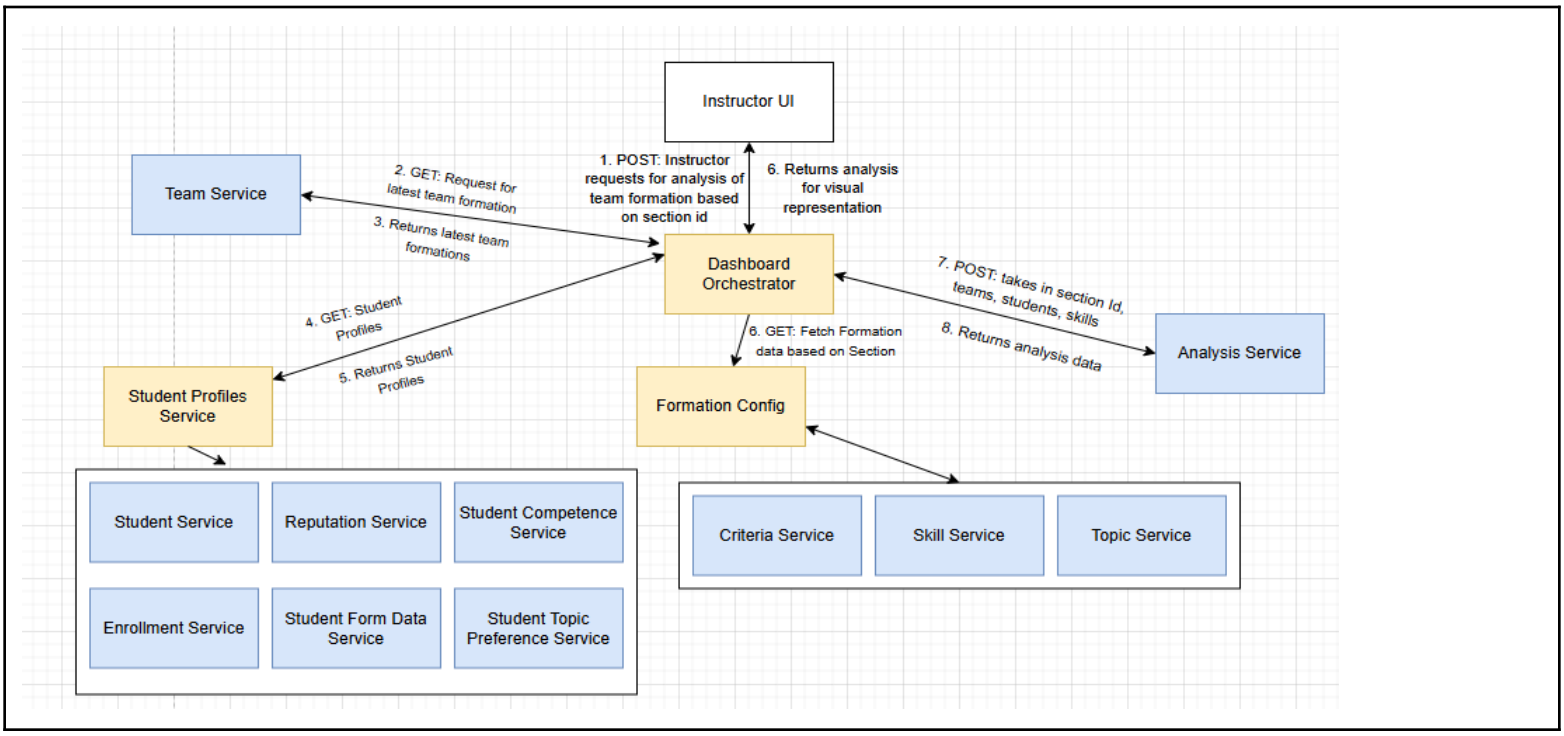


Fig 2 - Team Analytics for Instructor UI

1. *Instructor clicks on "view Analytics" and calls Dashboard Orchestrator based on section ID*
2. *Orchestrator will call from teams Service, Student Profile Service, Formation Config settings as well.*
3. *Data extracted by Dashboard Orchestrator is send to Analysis Service via POST to get analytics data*
4. *Dashboard Orchestrator outputs JSON to the instructor UI to display the analytics.*